

HDOC Day-25

C Programming - Decreasing Patterns using Nested Loops and Switch Case

Problem Statement

Write a menu-driven C program that uses a switch statement to print four decreasing patterns selected by the user. Nested loops are used without arrays, and each pattern is implemented in a separate parameterless function with no return value.

Menu used in the program

Choice	Pattern
1	Decreasing right-angled triangle
2	Decreasing natural-number pattern
3	Decreasing repeated-number pattern
4	Reverse-number triangle
5	Exit

Activity 1: Decreasing right-angled triangle

Prints a decreasing right-angled triangle of asterisks.

Algorithm

1. Start.
2. Read the number of rows n.
3. Set outer loop i from n down to 1.
4. For each row, run inner loop j from 1 to i and print an asterisk.
5. Move to the next line after each row.
6. Stop.

Test-case table

Test	Input n	Expected first row	Expected result
1	1	*	Pattern prints correctly with decreasing rows
2	4	* * * *	Pattern prints correctly with decreasing rows
3	6	* * * * * *	Pattern prints correctly with decreasing rows

Test-case execution

Test case 1 - choice 1, n = 1

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 1
Enter number of rows: 1
*
```

Test case 2 - choice 1, n = 4

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 1
Enter number of rows: 4
* * * *
* * *
* *
*
*
```

Test case 3 - choice 1, n = 6

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 1
Enter number of rows: 6
* * * * *
* * * *
* * *
* *
*
*
```

Activity 2: Decreasing natural-number pattern

Prints natural numbers starting from 1, with one fewer value in each next row.

Algorithm

1. Start.
2. Read the number of rows n.
3. Set outer loop i from n down to 1.
4. For each row, print j from 1 to i using the inner loop.
5. Print a new line after every row.
6. Stop.

Test-case table

Test	Input n	Expected first row	Expected result
1	1	1	Pattern prints correctly with decreasing rows
2	4	1 2 3 4	Pattern prints correctly with decreasing rows
3	5	1 2 3 4 5	Pattern prints correctly with decreasing rows

Test-case execution

Test case 1 - choice 2, n = 1

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 2
Enter number of rows: 1
1
```

Test case 2 - choice 2, n = 4

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 2
Enter number of rows: 4
1 2 3 4
1 2 3
1 2
1
```

Test case 3 - choice 2, n = 5

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 2
Enter number of rows: 5
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Activity 3: Decreasing repeated-number pattern

Starts from n and prints each row value as many times as the value itself.

Algorithm

1. Start.
2. Read the number of rows n.
3. Set outer loop i from n down to 1.
4. Print the current value i exactly i times using the inner loop.
5. Move to a new line and continue with i - 1.
6. Stop.

Test-case table

Test	Input n	Expected first row	Expected result
1	1	1	Pattern prints correctly with decreasing rows
2	4	4 4 4 4	Pattern prints correctly with decreasing rows
3	5	5 5 5 5 5	Pattern prints correctly with decreasing rows

Test-case execution

Test case 1 - choice 3, n = 1

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 3
Enter number of rows: 1
1
```

Test case 2 - choice 3, n = 4

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 3
Enter number of rows: 4
4 4 4 4
3 3 3
2 2
1
```

Test case 3 - choice 3, n = 5

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 3
Enter number of rows: 5
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Activity 4: Reverse-number triangle

Prints each row in reverse order, beginning with the number of elements in that row and ending at 1.

Algorithm

1. Start.
2. Read the number of rows n.
3. Set outer loop i from n down to 1.
4. For each row, set j = i and print j while decreasing it to 1.
5. Print a new line after every row.
6. Stop.

Test-case table

Test	Input n	Expected first row	Expected result
1	1	1	Pattern prints correctly with decreasing rows
2	4	4 3 2 1	Pattern prints correctly with decreasing rows
3	5	5 4 3 2 1	Pattern prints correctly with decreasing rows

Test-case execution

Test case 1 - choice 4, n = 1

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 4
Enter number of rows: 1
1
```

Test case 2 - choice 4, n = 4

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 4
Enter number of rows: 4
4 3 2 1
3 2 1
2 1
1
```

Test case 3 - choice 4, n = 5

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 4
Enter number of rows: 5
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

Complete C Program

The same menu-driven program is used for all four activities. Each case calls a separate void function, so the code stays simple and easy to test.

```
#include <stdio.h>

void pattern1();
void pattern2();
void pattern3();
void pattern4();

int main()
{
    int choice;

    printf("1. Decreasing right-angled triangle\n");
    printf("2. Decreasing natural-number pattern\n");
    printf("3. Decreasing repeated-number pattern\n");
    printf("4. Reverse-number triangle\n");
    printf("5. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1: pattern1(); break;
        case 2: pattern2(); break;
        case 3: pattern3(); break;
        case 4: pattern4(); break;
        case 5: printf("Program ended.\n"); break;
        default: printf("Invalid choice.\n");
    }
    return 0;
}

void pattern1()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for(i = n; i >= 1; i--) {
        for(j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
}

void pattern2()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for(i = n; i >= 1; i--) {
        for(j = 1; j <= i; j++) printf("%d ", j);
        printf("\n");
    }
}

void pattern3()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for(i = n; i >= 1; i--) {
        for(j = 1; j <= i; j++) printf("%d ", i);
        printf("\n");
    }
}

void pattern4()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for(i = n; i >= 1; i--) {
        for(j = i; j >= 1; j--) printf("%d ", j);
        printf("\n");
    }
}
```

Compilation and Execution

Save the file as **day25.c**. Compile it in Kali Linux using **gcc day25.c -o a.out**, then run it using **./a.out**. Select a menu choice and enter the required number of rows.

Result

The menu-driven C program was written and tested successfully. All four decreasing patterns were produced correctly for the prepared test cases using nested loops, switch-case selection and separate parameterless void functions.